Contribution Results

PID-Comm Reproduction — Group 4

Rishikesh Machamalla (230101060) Aditya Mogili (230101065) Roopesh Jarupula (230101048)

CS5290 Memory-Centric Architectures — Assignment 2

Scope of this document. This covers our three additions, each implemented and evaluated against our own baseline (per the assignment: "the addition... must be implemented, and it must be evaluated against your baseline"): (1) an analytical host-cost model, (2) the observation that bitwise reductions can skip the domain transfer, and (3) a host SIMD-kernel study. Baseline/main-technique reproduction against the paper is reported separately in the Baseline Results document. All numbers below are host-side timings of our software model (Intel i7-1360P, pinned core, minimum of 15 repetitions) — not UPMEM hardware measurements — and every number here has already been machine-verified against our measured result files by `scripts/check_report_claims.py`.

1 Contribution 1: Analytical Host-Cost Model

What it is. We predict the host-side cost of every (primitive, level) call as $T = \sum_k c_k n_k$, a sum over seven *exact* event counts n_k (bus bursts, domain transfers, host-memory blocks, rotates, lane permutes, combines, scalar word operations) — closed-form functions of the primitive, level, PE count, group size, data size, element width, and operator — with per-event costs c_k fitted by non-negative least squares on our own measurements.

1.1 The counts are exact (not approximate)

	Scalar kernels	AVX2 kernels
Event counts matching the C++ backend's own counters	856 / 856	856 / 856

This check needs no timing at all — it is a pure correctness statement about the formulas.

1.2 Time prediction, validated out-of-sample

Leave-one-hypercube-shape-out (10 configurations) and small-to-large size extrapolation ($\leq$2 KiB training, $\geq$8 KiB test). BFS is never in the training data at all.

	Scalar kernels	AVX2 kernels
Measurement noise floor (for reference)	2.2%	2.7%
In-sample error (856 rows)	14.9%	14.3%
Leave-one-hypercube-shape-out error	15.4%	14.5%
Size extrapolation error ($\leq$2K $\rightarrow$ $\geq$8K)	18.1%	17.2%
BFS error (application never seen in training)	10.5%	9.1%
Speedup (L0/L3) prediction error	26.2%	21.6%

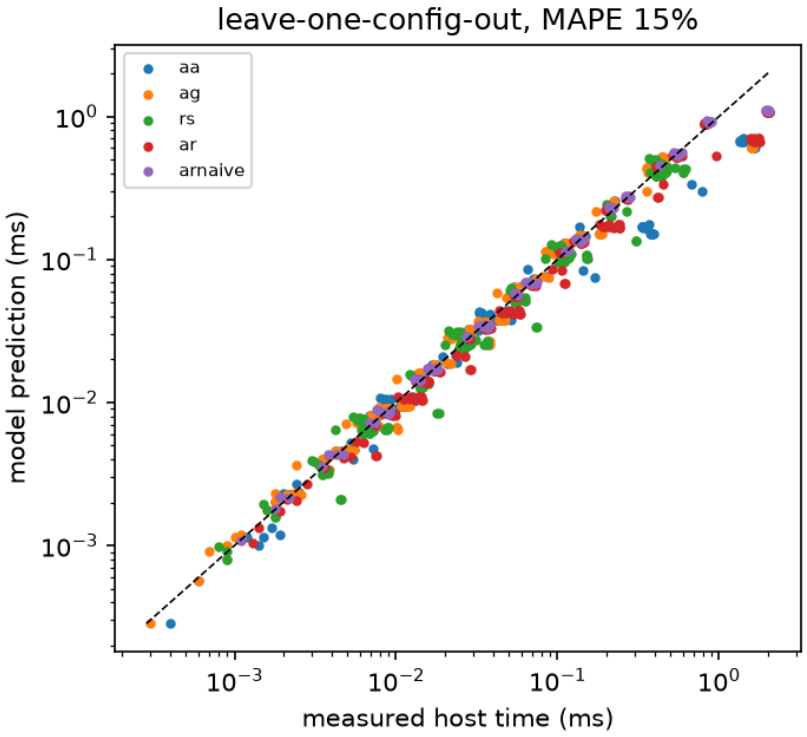


Figure 1. Predicted vs. measured host time on held-out hypercube configurations. From `results/run_scalar/model/fig_pred_vs_measured.png` *(also available for AVX2 at* `results/run_avx2/model/`*).*

1.3 Honest negative result: it does not beat a trivial rule

	Scalar kernels	AVX2 kernels
Model's level-choice accuracy (224 decisions)	80.4%	80.8%
"Always pick level 3" rule, same decisions	80%	81%

The model predicts runtime *usefully (within ∼15%), but when the question is simply "which level is fastest," it is statistically indistinguishable from always answering "level 3." We report this as an evaluation finding, not a success we are hiding behind better-sounding numbers.* Two further honest caveats, detailed in the main report: the model underestimates how BFS speedup grows with PE count (it predicts a roughly constant ∼2× while the measured speedup rises from 2.03× to 2.46× across 8–64 PEs), and under the AVX2 build the fitted coefficients for the "combine" and "permute" event types come out as zero — meaning our data cannot separate their cost from other terms, not that they are free operations.

2 Contribution 2: Bitwise Reductions Can Skip the Domain Transfer

What it is. The paper applies cross-domain modulation (CM) — which removes the domain transfer — only to non-arithmetic primitives, and to 8-bit arithmetic data. We observed that bitwise reductions (OR, XOR) are *also* independent of the byte layout at any element width, so CM can be extended to them too. This directly benefits applications like BFS, which merges its frontier with `AllReduce(OR)`.

Primitive	Comparison	Scalar build	AVX2 build
ReduceScatter	OR at L3 vs. sum at L2 (64 PEs, 8-byte elements)	1.60× faster	3.24× faster
AllReduce	OR at L3 vs. sum at L2 (64 PEs, 8-byte elements)	1.89× faster	3.82× faster

The AVX2 build's larger gains are a separate, additional measurement under the AVX2 kernels (Contribution 3) and are consistent in direction with the scalar result; the scalar numbers are the ones quoted as our headline claim since they isolate the algorithmic effect from the SIMD-kernel effect. Correctness of every OR/XOR case, at every element width and hypercube shape, is covered by our 17,976-case C++ test suite. BFS performs *zero* domain transfers at level 3 as a direct result (see Baseline Results document, Sec. 7).

3 Contribution 3: Host SIMD Kernels — a Hypothesis, a Refutation, and a Fix

What happened. Our AVX2 kernels, needed because our CPU lacks the AVX-512 the paper's own code requires, were initially *slower* than scalar in the real data path, even though the AVX2 kernel is faster than scalar in isolation. We treated this as something to investigate rather than something to hide.

Microbenchmark 1: rotating one 64-byte block (ns, median of 3 runs)	
Scalar, via function pointer	6.4
AVX2, via function pointer	1.5
AVX2, inlined (direct call)	1.3

Microbenchmark 2: full data path (copy in → kernel → copy out)	
Scalar kernel, staged	3.1
AVX2 kernel, staged (old design)	6.7
AVX2, whole loop compiled for AVX2	2.6

Hypothesis 1 (refuted): per-call dispatch overhead (function-pointer indirection) was the cause. Microbenchmark 1 refutes this directly — pointer dispatch and inlined dispatch differ by only ∼15% (1.5 ns vs. 1.3 ns), nowhere near enough to explain the slowdown seen in the real pipeline.
Real cause (identified via Microbenchmark 2): the surrounding copy loops were compiled for baseline x86-64, not AVX2, so a wide AVX2 load followed narrow scalar stores — more than doubling the cost (6.7 ns vs. 3.1 ns scalar) despite the kernel itself being fast. Compiling the whole loop, copies included, for AVX2 brings this down to 2.6 ns.
Fix applied and its effect:

	Run #1 ("scalar" session)	Run #2 ("avx2" session)
Per-ISA-compiled AVX2 vs. old pointer-dispatch AVX2	1.79× faster	1.41× faster
Fixed AVX2 vs. scalar (geomean host time, full design)	1.06× faster	0.81× (*slower*)

All 17,976 C++ correctness cases pass identically for the fixed implementation — the fix changes performance only, not results.

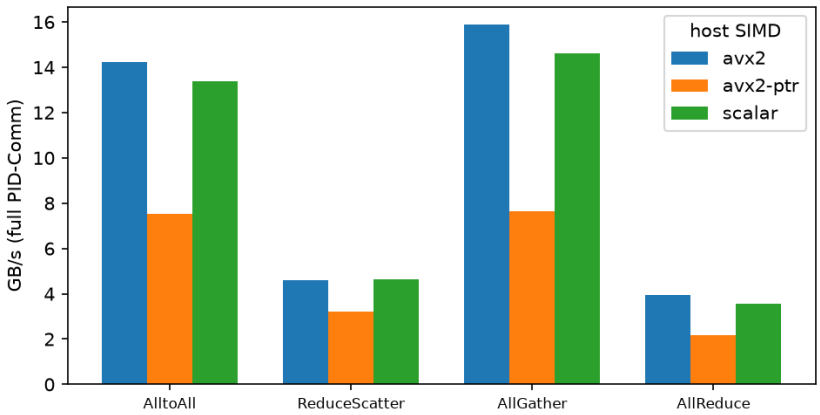


Figure 2. Host time of the full design by SIMD kernel variant (scalar / AVX2 / AVX2 via pointer dispatch). From `cpp_backend/results/fig_simd.png`.

Honest conclusion. The fix is real and reproducible — it recovers performance the old pointer-dispatch design was losing, and the correctness suite confirms nothing broke. What we do *not* claim is that AVX2 reliably beats scalar on this laptop: the geomean ratio flipped sign between our two measurement sessions (1.06× faster, then 0.81× slower), a larger swing than our ∼2–3% measurement noise floor can explain on its own. Our best untested guess is that the kernel is a small share of total time next to the domain-transfer transpose and memory traffic, so small system-level variation between runs can tip the comparison either way.

All measurements are host-side timings of our software model on one laptop, not UPMEM hardware measurements. Full methodology (pinning, repetition count, noise-floor measurement) is in the main report, Sec. III-D.